Image Denoising Using Convolutional Autoencoders

A Course End Project Report Submitted in the Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

Computer Science and Engineering(Data Science)

Deep Learning (A8707)

Submitted By

23881A67C7	**A.Sri Sumukhi**
23881A67E5	**G.Yogitha**
24885A6719	**K.Sravani**

Under the Esteemed Guidance of

M.Bhuvaneswari

Assistant Professor

VARDHAMAN
COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(DATA SCIENCE)

VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Affiliated to JNTUH, Approved by AICTE and Accredited by NBA)
Shamshabad, Hyderabad-501218

2025-2026

VARDHAMAN
COLLEGE OF ENGINEERING

VARDHAMAN COLLEGE OF ENGINEERING

Affiliated to **JNTUH**, Approved by **AICTE**, Accredited by **NAAC**, with **A++** Grade, **ISO 9001:2015** Certified. Kacharam, Shamshabad, Hyderabad – 501218, Telangana, India

Department of Computer Science and Engineering(Data Science)

CERTIFICATE

This is to certify that the Course End Report work entitled **"Image Denoising using Convolutional Autoencoders"** carried out by A.Sri Sumukhi, Roll Number **23881A67C7**, G.Yogitha, Roll Number **23881A67E5,** K.Sravani, Roll Number **24885A6719** towards Course-End Project and submitted to the Department of Computer Science and Engineering(Data Science) in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering(Data Science)** during the year 2025-26

Name & Signature of the Instructor	Name & Signature of the HoD, CSD
M.Bhuvaneswari	Dr.Shanthi Makka

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown allthe efforts with success.

We wish to express my deep sense of gratitude to **M. Bhuvaneswari,** Assistant Professor for their able guidance and useful suggestions, which helped us in completing the design part of potential project in time.

We are particularly thankful to **Dr. Shanthi Makka, HOD(CSD)** intense support and encouragement, which helped us to mould our project into a successful one.

We show gratitude to our honorable Principal **Prof.J.V.R.Ravindra**, for having provided all the facilities and support.

We avail this opportunity to express our deep sense of gratitude and heartful thanks to **Dr. Teegala Vijender Reddy**, Chairman and **Sri Teegala Upender Reddy**, Secretary of VCE, for providing congenial atmosphere to complete this project successfully.

We also thank all the staff members of the department of **CSD** for their valuable support and generous advice. Finally, thanks to all our friends and family members for their continuous support and enthusiastichelp.

Table of Contents

1. Abstract	1-1
2. Introduction	2-3
2.1 Problem Statement	2-2
2.2 Objective of the Project	2-2
2.3 Importance / Business Value	3-3
2.4 Scope and Limitations	3-3
3. Data Collection	4-4
3.1 Data Sources	4-4
3.2 Data Description	4-4
3.3 Data Access Methods	4-4
4. Data Cleaning & Preprocessing	5-5
4.1 Handling Missing Values	5-5
4.2 Removing Duplicates	5-5
4.3 Data Type Conversions	5-5
4.4 Outlier Detection and Treatment	5-5
4.5 Feature Engineering	5-5
5. Exploratory Data Analysis (EDA)	6-6
5.1 Univariate Analysis	6-6
5.2 Bivariate / Multivariate Analysis	6-6
5.3 Visualizations	6-6
5.4 Insights Drawn	6-6
6. Statistical Analysis	7-7
7. Model Building	7-8
7.1 Problem Type	7-7
7.2 Model Selection	8-8
7.3 Training and Testing Split	8-8
7.4 Evaluation Metrics	8-8
7.5 Cross Validation	8-8
8. Results & Interpretation	8-9
9. Tools & Technologies Used	10-10
10. Conclusion	11-11
11. References	12-12

1. Abstract

Image denoising is a crucial task in computer vision that aims to remove noise from corrupted images while preserving important visual details. Noisy images often arise due to environmental conditions, sensor limitations, or transmission errors, which can significantly affect the performance of image processing and machine learning systems. The model is trained on the MNIST dataset, where artificial Gaussian noise is added to clean images to create noisy inputs. The autoencoder learns compressed representations of images and reconstructs them through a decoding process, effectively removing the noise.The model's performance is evaluated using quantitative metrics such as Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM). Visual comparisons between original, noisy, and reconstructed images are also used to assess the effectiveness of the model. The results demonstrate that autoencoders can successfully reduce noise and restore image quality, making them useful for applications in medical imaging, surveillance, photography, and image restoration.

In addition to improving image quality, this project demonstrates the effectiveness of deep learning models in solving real-world image restoration problems. The proposed autoencoder architecture learns compressed feature representations of images and reconstructs them by minimizing reconstruction loss during training. By comparing original, noisy, and reconstructed images, the model shows its ability to preserve essential structures while reducing noise artifacts. The experimental results highlight the potential of autoencoders as a powerful tool for preprocessing images before performing advanced computer vision tasks such as object detection, image classification, and pattern recognition. This approach can be further extended to handle different types of noise and higher-resolution datasets, making it a valuable solution for various applications in computer vision and image processing systems.

2. Introduction

With the rapid advancement of digital imaging technologies, images have become a fundamental medium for communication, data analysis, and automation. However, images captured by cameras or sensors often contain noise due to environmental conditions, hardware limitations, or transmission errors. Noise reduces the quality of images and negatively affects computer vision systems that rely on accurate visual data.Traditional image denoising techniques such as Gaussian filtering, median filtering, and Wiener filtering have been widely used to reduce noise. While these methods can remove certain types of noise, they often result in loss of image details and blurring of edges.Recent advancements in **deep learning** have introduced powerful models capable of learning complex patterns from data. One such model is the **Autoencoder**, a neural network architecture designed for learning compressed representations of data. Autoencoders can be trained to reconstruct input data while removing unwanted noise.

In this project, a **Convolutional Autoencoder** is implemented to learn meaningful features from noisy images and reconstruct clean images. The model is trained on the MNIST dataset with artificially added noise. The performance of the model is evaluated using quantitative metrics and visual comparisons.

2.1 Problem Statement

Images captured by cameras or sensors are often affected by noise due to factors such as low lighting conditions, hardware limitations, or transmission errors. Noise degrades image quality and can negatively impact computer vision tasks such as object recognition, image classification, and medical diagnosis. Traditional filtering techniques may remove noise but often blur important details. Therefore, there is a need for intelligent techniques that can remove noise while preserving the structural integrity of the image.
Noisy images can significantly degrade the performance of machine learning systems used for:
-Object detection
-Face recognition
-Medical diagnosis
-Autonomous vehicles

Therefore, there is a need for an intelligent system capable of effectively removing noise from images while preserving essential details and structures.

2.2 Objective of the Project

The main objective of this project is to develop an autoencoder-based deep learning model capable of removing noise from corrupted images and reconstructing clean versions of those images. The project focuses on training the model using noisy images as input while using the corresponding clean images as the target output, enabling the model to learn meaningful image features and patterns. Another important objective is to evaluate the performance of the model using both quantitative metrics and visual analysis to determine how effectively the noise has been removed. Through this approach, the project aims to demonstrate the capability and effectiveness of autoencoders as powerful deep learning techniques for image restoration and image quality enhancement tasks.

2.3 Importance/Business Value

Image denoising has numerous real-world applications across various fields where image quality is crucial for accurate analysis and decision-making. In the field of medical imaging, denoising techniques help enhance MRI and CT scan images, enabling doctors to make more accurate diagnoses. In satellite imaging, improving the clarity of satellite images supports weather forecasting, environmental monitoring, and geographical analysis. Surveillance **systems** also benefit from image denoising, as clearer security camera footage helps in better object detection and identification. In the photography and media industry, denoising techniques improve image quality, especially in low-light conditions where images often appear grainy. Additionally, in automated vision systems, high-quality images are essential for improving the performance of machine learning models used in applications such as object recognition, autonomous vehicles, and intelligent monitoring systems.

2.4 Scope and Limitations

Scope:

The scope of this project focuses on developing and implementing a deep learning model for removing noise from images using an autoencoder architecture. The model is trained using a publicly available dataset, specifically the MNIST dataset, which provides a large collection of handwritten digit images suitable for machine learning experiments. The project involves preprocessing the dataset, adding artificial noise to the images, training the model to learn meaningful representations, and reconstructing clean images from noisy inputs. The performance of the model is then evaluated using multiple quantitative metrics and visual analysis to determine how effectively the noise has been removed and the image quality has been restored.

Limitations:

Despite the effectiveness of the model, this project has certain limitations. The model is trained only on grayscale images from the MNIST dataset, which are relatively simple and low in resolution compared to real-world images. As a result, the model's performance may vary when applied to high-resolution or complex images containing different textures and colors. Additionally, the project considers only Gaussian noise for simulating noisy conditions, whereas real-world images may contain different types of noise such as salt-and-pepper noise, speckle noise, or motion blur. These limitations suggest that further improvements and experiments with larger datasets and different noise types may be required to enhance the model's generalization capability.

3. Data Collection

The data used in this project was obtained from the MNIST dataset, which is a widely used benchmark dataset in machine learning and computer vision research. The dataset consists of grayscale images of handwritten digits ranging from 0 to 9, collected from different individuals to provide a diverse representation of writing styles. Each image in the dataset has a resolution of 28 × 28 pixels, making it suitable for training deep learning models while keeping computational requirements relatively low. The dataset contains a total of 70,000 images, of which 60,000 are used for training the model and 10,000 are used for testing and evaluation. The MNIST dataset is available through the TensorFlow and Keras libraries, allowing easy access and integration into Python-based machine learning workflows. For the purpose of this project, the dataset was loaded directly using the Keras API, and artificial noise was added to the images to simulate real-world noisy conditions required for training the image denoising autoencoder model.

3.1 Data Sources

The dataset used in this project is the **MNIST dataset**, which contains a large collection of handwritten digit images that are widely used for machine learning and deep learning experiments. This dataset consists of grayscale images representing digits from 0 to 9 and is commonly used as a benchmark for testing image processing and neural network models. The MNIST dataset is easily accessible through the built-in datasets provided by the **TensorFlow/Keras library**, which allows it to be directly loaded into the programming environment without the need for manual downloading. Using this dataset enables efficient training and evaluation of the autoencoder model for image denoising tasks.

3.2 Data Description

The MNIST dataset consists of a total of 70,000 handwritten digit images, which are divided into two main sets: 60,000 images for training the model and 10,000 images for testing and evaluation. Each image in the dataset has a size of 28 × 28 pixels and is represented in grayscale format, meaning it contains only intensity values rather than color information. The pixel values in each image range from 0 to 255, where lower values represent darker pixels and higher values represent lighter pixels. Each image corresponds to a handwritten digit ranging from 0 to 9, written by different individuals, providing a diverse set of digit patterns for training and testing machine learning models.

3.3 Data Access Methods

The MNIST dataset used in this project is accessed using the TensorFlow Keras API, which provides a convenient way to load commonly used datasets for machine learning tasks. By importing the dataset through the Keras library using the command `from tensorflow.keras.datasets import mnist`, the dataset can be easily loaded into the working environment. The function `mnist.load_data()` is then used to retrieve the dataset, which automatically downloads the data if it is not already available locally and splits it into training and testing sets. This method simplifies the process of obtaining the dataset and allows it to be directly used for preprocessing, model training, and evaluation within the Python programming environment.

4. Data Cleaning & Preprocessing

Before training the model, the dataset undergoes several preprocessing steps to ensure it is suitable for machine learning and deep learning operations. Data preprocessing is an essential step that prepares the dataset for efficient model training and improves the performance of the neural network. In this project, **image normalization** is first performed where pixel values, which originally range from 0 to 255, are scaled to a range between 0 and 1 to improve training stability and help the neural network converge faster during learning. Next, the images are reshaped into a four-dimensional tensor format required for convolutional neural networks, following the structure *(number_of_samples, height, width, channels)*. For the MNIST dataset, this results in a shape of *(60000, 28, 28, 1)*. Finally, artificial Gaussian noise is added to the images to simulate real-world noisy conditions. This noise is generated using a random normal distribution and added to the pixel values of the images. The noisy images are used as the input to the autoencoder model, while the original clean images serve as the target output during training, enabling the model to learn how to remove noise and reconstruct clear images.

4.1 Handling Missing Values

The MNIST dataset does not contain missing values. Therefore, no additional handling was required.

4.2 Removing Duplicates

The dataset provided is already curated and does not contain duplicate images.

4.3 Data Type Conversions

The image pixel values were converted from integers to floating-point numbers and normalized to the range 0–1.

Example:

```
x_train = x_train.astype('float32') / 255
```

4.4 Outlier Detection and Treatment

Since the dataset contains standardized digit images, no significant outliers were found.

4.5 Feature Engineering

Noise was artificially added to images to simulate real-world corrupted images. Gaussian noise was introduced using random distributions.

Example:

```
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
```

This created the noisy dataset used for training the denoising model.

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the structure and characteristics of the dataset before building the model. Initially, several sample images from the MNIST dataset were visualized to observe the handwritten digit patterns and variations in writing styles. After adding artificial Gaussian noise to the images, a comparison between original and noisy images was conducted to understand how noise affects pixel intensity and image clarity. Histograms and visual plots were used to analyze the distribution of pixel values across the dataset. These visualizations helped identify how noise changes the pixel distribution and increases variability in the images. Additionally, side-by-side image comparisons were performed to examine the differences between clean, noisy, and reconstructed images. The insights obtained from EDA helped in understanding the dataset behavior and confirmed the necessity of using a deep learning model to effectively remove noise while preserving the important features of the images.EDA helps understand the characteristics of the dataset.

5.1 Univariate Analysis:

Univariate analysis was conducted to examine the distribution of individual features in the dataset, particularly the pixel intensity values of the images. By analyzing each image independently, the brightness variations and pixel value patterns within the dataset were better understood. Histograms were used to visualize the distribution of pixel intensities, which helped identify how pixel values are spread across the images. This analysis provided insight into the overall characteristics of the dataset and helped understand the effect of noise on pixel brightness levels.

5.2 Bivariate / Multivariate Analysis:

Bivariate and multivariate analysis were performed to study the relationships between the original clean images and the noisy images. This analysis helped in understanding how the addition of noise affects pixel intensities and the structural patterns within the images. By comparing pixel values between the clean and noisy images, it was possible to observe the changes in intensity and distortion caused by noise, which justified the need for a model capable of restoring image quality.

5.3 Visualizations:

Various visualizations were created to better understand the dataset and evaluate the model's performance. These visualizations included displaying original images, noisy images, and reconstructed images produced by the autoencoder. Different types of plots such as histograms, image grids, and pixel distribution graphs were used to visually analyze the differences between clean and noisy data and to observe how effectively the model reconstructed the images after removing noise.

5.4 Insights Drawn:

Several important insights were obtained from the exploratory analysis. It was observed that noise significantly alters the pixel intensity values of images, causing distortion and reducing image clarity. Clean images generally show well-structured patterns representing handwritten digits, while noisy images appear blurred or distorted due to random variations in pixel values. The analysis also indicated that autoencoder models are capable of learning meaningful feature representations from noisy images and can effectively reconstruct cleaner versions of the images by reducing noise while preserving important structural details.

6. Statistical Analysis

Statistical analysis was conducted to understand the numerical characteristics of the image dataset and to evaluate the quality of reconstructed images produced by the autoencoder model. Basic summary statistics such as mean, variance, and standard deviation of pixel intensity values were calculated to analyze the distribution of image data before and after adding noise. These statistics helped in understanding how noise affects the overall pixel distribution and variability in the images. Additionally, quantitative evaluation metrics such as Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM) were used to measure the similarity between original and reconstructed images. MSE measures the average squared difference between the original and reconstructed pixel values, while PSNR evaluates the ratio between signal strength and noise level. SSIM measures the perceptual similarity between images by considering luminance, contrast, and structural information. These statistical measures provide an objective assessment of the model's performance in removing noise and reconstructing high-quality images.

In this project, basic statistical measures such as mean, standard deviation, and variance were calculated to understand the distribution of pixel intensity values in the image dataset. The mean pixel value represents the average brightness level of the pixels in the images and is calculated by summing all pixel values and dividing by the total number of pixels in the dataset. The standard deviation measures how much the pixel values vary from the mean, indicating the level of variation or spread in brightness across the images; higher standard deviation means greater variation in pixel intensity. The variance represents the squared deviation of pixel values from the mean and provides another measure of dispersion within the dataset. In this project, these statistical measures help analyze how noise affects pixel distributions and allow comparison between the original clean images and the noisy images used for training the autoencoder model.

7. Model Building

Model building in this project involved designing and implementing a Convolutional Autoencoder to perform image denoising. The autoencoder consists of two main components: an encoder and a decoder. The encoder part of the network compresses the input noisy images into a lower-dimensional feature representation by using convolutional and pooling layers, which helps the model capture important patterns and structures present in the images. The decoder then reconstructs the original image from this compressed representation using convolutional and upsampling layers. During training, the noisy images are provided as input to the model, while the clean images are used as the target output. The model learns to minimize the reconstruction error between the original and reconstructed images using an appropriate loss function such as binary cross-entropy or mean squared error. The dataset is divided into training and testing sets to evaluate the model's ability to generalize to unseen data. Through multiple training epochs, the autoencoder gradually learns to remove noise and produce clearer reconstructed images.

7.1 Problem Type

This is a **deep learning reconstruction problem** using unsupervised learning.

Type: Image Denoising using Autoencoder.

7.2 Model Selection

A Convolutional Autoencoder architecture was selected for this project because it is highly effective in processing image data and capturing important spatial features present in images. Convolutional layers help the model learn patterns such as edges, shapes, and textures while maintaining the spatial relationships between pixels. At the same time, the autoencoder reduces the dimensionality of the input data through the encoding process while preserving the essential structures of the image. This compressed representation allows the decoder to reconstruct the image while removing unwanted noise. Due to its ability to learn meaningful feature representations and accurately reconstruct images, the Convolutional Autoencoder is widely used in image reconstruction and image denoising tasks in deep learning applications.

7.3 Training and Testing Data Split

In this project, the MNIST dataset was divided into training and testing sets to build and evaluate the autoencoder model. The training dataset consists of **60,000 images**, which are used to train the model to learn patterns and features from the data. The testing dataset contains 10,000 images, which are used to evaluate the model's performance on unseen data. During training, noisy images are provided as the input to the model, while the corresponding clean images are used as the target output. This approach enables the autoencoder to learn how to remove noise from corrupted images and reconstruct clearer versions of the original images.

7.4 Evaluation Metrics

Model performance was evaluated using:

Mean Squared Error (MSE)
Peak Signal-to-Noise Ratio (PSNR)
Structural Similarity Index (SSIM)

7.5 Cross Validation

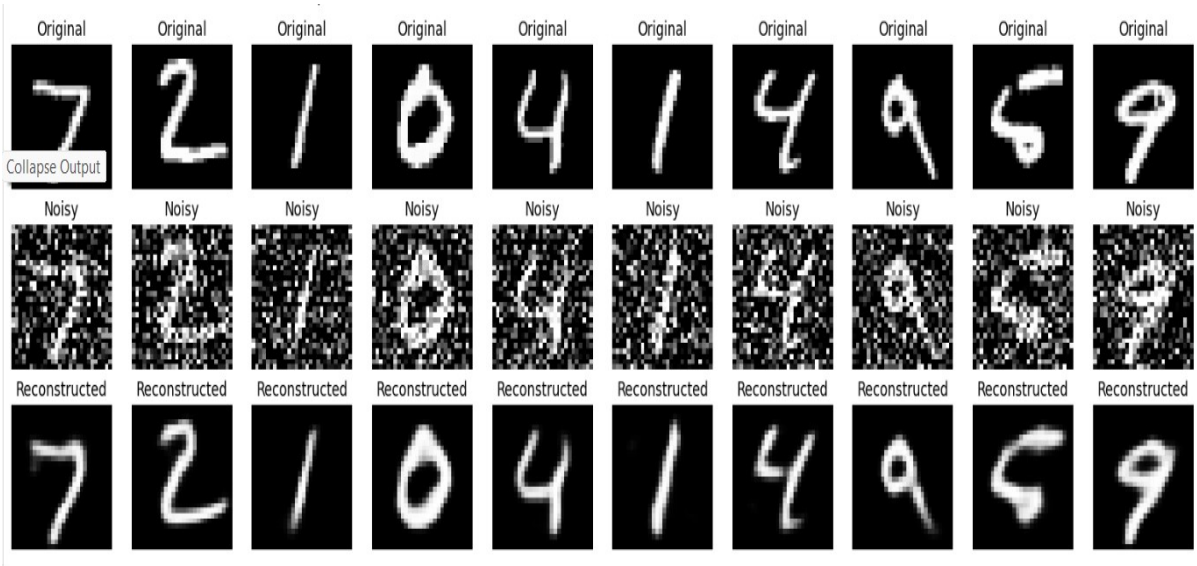
Due to the large dataset size and computational requirements, traditional cross-validation was not used. Instead, a training-validation split was applied.

8. Results & Interpretation

The trained autoencoder model successfully reconstructed clean images from noisy inputs, demonstrating its effectiveness in removing noise while preserving important image features. The reconstructed images showed a significant reduction in noise and closely resembled the original clean images, indicating that the model was able to learn meaningful representations of the image data. The performance of the model was evaluated using quantitative metrics such as Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM). A low MSE value indicated minimal reconstruction error, while a high PSNR value reflected strong signal reconstruction quality. Similarly, high SSIM values confirmed a high level of similarity between the original and reconstructed images. In addition to these quantitative results, visual comparisons between original, noisy, and reconstructed images clearly demonstrated the effectiveness of the model. The outcomes of this project highlight important practical applications, including improving image quality in medical imaging, surveillance and security systems, and digital photography.

Model Results

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be
used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 30s 54ms/step - loss: 0.1823 - val_loss: 0.1240
Epoch 2/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 39s 51ms/step - loss: 0.1199 - val_loss: 0.1144
Epoch 3/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 25s 52ms/step - loss: 0.1134 - val_loss: 0.1099
Epoch 4/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 24s 52ms/step - loss: 0.1096 - val_loss: 0.1071
Epoch 5/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 24s 52ms/step - loss: 0.1072 - val_loss: 0.1053
Epoch 6/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 25s 53ms/step - loss: 0.1057 - val_loss: 0.1039
Epoch 7/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 25s 52ms/step - loss: 0.1045 - val_loss: 0.1032
Epoch 8/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 27s 57ms/step - loss: 0.1037 - val_loss: 0.1022
Epoch 9/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 31s 65ms/step - loss: 0.1030 - val_loss: 0.1022
Epoch 10/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 26s 56ms/step - loss: 0.1025 - val_loss: 0.1015
Epoch 11/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 27s 58ms/step - loss: 0.1020 - val_loss: 0.1012
Epoch 12/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 28s 60ms/step - loss: 0.1015 - val_loss: 0.1004
Epoch 13/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 28s 60ms/step - loss: 0.1012 - val_loss: 0.1001
Epoch 14/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 32s 68ms/step - loss: 0.1009 - val_loss: 0.1001
Epoch 15/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 29s 62ms/step - loss: 0.1006 - val_loss: 0.0995
Epoch 16/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 31s 66ms/step - loss: 0.1004 - val_loss: 0.0995
Epoch 17/20
```



9. Tools & Technologies Used

Various tools and technologies were used in this project to implement the image denoising model effectively. The primary programming language used was Python, due to its extensive support for machine learning and data analysis libraries. The project utilized several Python libraries such as NumPy for numerical computations and array operations, Matplotlib for data visualization and image plotting, and TensorFlow with Keras for building and training the deep learning autoencoder model. Additionally, the Scikit-image library was used to calculate evaluation metrics such as Structural Similarity Index (SSIM) and Peak Signal-to-Noise Ratio (PSNR). The development and experimentation were performed using Jupyter Notebook, which provides an interactive environment for writing and executing code while visualizing outputs. In some cases, Google Colab can also be used to run the model with cloud-based computational resources and GPU support. These tools and technologies collectively helped in data preprocessing, model development, training, evaluation, and visualization of the results.

Programming Language

Python: This project was implemented using a combination of powerful programming tools, libraries, and development platforms that support machine learning and deep learning applications. The primary programming language used for the development of this project is Python, which is widely recognized for its simplicity, flexibility, and extensive ecosystem of libraries for data science, artificial intelligence, and image processing tasks. Python provides efficient support for numerical computations, data analysis, and neural network implementation, making it an ideal choice for developing deep learning models such as autoencoders.

Libraries

Several specialized libraries were used to support different stages of the project. **NumPy** was used for performing numerical operations and handling large multi-dimensional arrays, which are essential for processing image data. **Matplotlib** was utilized for data visualization.

10. Conclusion

The project successfully demonstrated the application of a Convolutional Autoencoder for removing noise from corrupted images and reconstructing clean versions of those images. By training the model on the MNIST dataset with artificially added noise, the autoencoder was able to learn meaningful features and effectively reduce noise while preserving the important structures of the images. The performance of the model was evaluated using quantitative metrics such as Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index (SSIM), along with visual comparisons between original, noisy, and reconstructed images. The results showed that the model was capable of producing high-quality reconstructed images with improved clarity and reduced distortion. This project highlights the potential of deep learning techniques in image restoration tasks and demonstrates how autoencoders can be used in practical applications such as medical imaging, surveillance systems, satellite imagery, and digital photography.Furthermore, the project provided valuable insights into how neural networks can automatically learn patterns and representations from data without manual feature extraction. The successful implementation of the convolutional autoencoder shows that deep learning models can effectively handle image reconstruction problems and improve image quality even when the input data contains significant noise. The approach used in this project can also be extended to other computer vision applications such as image compression, anomaly detection, and image enhancement. In future work, the model can be improved by using larger and more complex datasets, experimenting with deeper network architectures, and applying advanced techniques such as Variational Autoencoders (VAE) or Generative Adversarial Networks (GANs). Additionally, testing the model on real-world noisy images and high-resolution datasets would further enhance its practical usability and generalization capability in real-life scenarios.

11. References

- **LeCun, Y., Cortes, C., & Burges, C. J. C.** (1998). *The MNIST database of handwritten digits*. AT&T Labs. Available at: http://yann.lecun.com/exdb/mnist/
- **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep Learning*. MIT Press, Cambridge, MA. Available at: https://www.deeplearningbook.org/
- **Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A.** (2008). Extracting and composing robust features with denoising autoencoders. *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103.
- **TensorFlow Development Team.** (2023). *TensorFlow: An end-to-end open-source platform for machine learning*. Available at: https://www.tensorflow.org/
- **Pedregosa, F., Varoquaux, G., Gramfort, A., et al.** (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- **Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P.** (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612.
- **Hinton, G. E., & Salakhutdinov, R. R.** (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
- **Chollet, F.** (2021). *Deep Learning with Python* (2nd ed.). Manning Publications, Shelter Island, NY.
- **Bishop, C. M.** (2006). *Pattern Recognition and Machine Learning*. Springer, New York.